torch.mean#

https://pytorch.org/docs/stable/generated/torch.mean.html#torch.mean

Description:

If the input tensor is empty, torch.mean() returns nan. This behavior is consistent with NumPy and follows the definition that the mean over an empty set is undefined. Returns the mean value of all elements in the input tensor. Input must be floating point or complex. input (Tensor) – the input tensor, either of floating point or complex dtype dtype (torch.dtype, optional) – the desired data type of returned tensor. If specified, the input tensor is casted to dtype before the operation is performed. This is useful for preventing data type overflows. Default: None.

Function Signature

```
torch.mean(input, *, dtype=None) → Tensor#

Parameters

Keyword Arguments

torch.mean(input, dim, keepdim=False, *, dtype=None,
out=None) → Tensor

Parameters
```

Parameters

Keyword

dtype (torch.dtype, optional) – the desired data type of returned tensor. If specified, the input tensor is casted to dtype before the operation is performed. This is useful for preventing data type overflows. Default: None.

Keyword

dtype (torch.dtype, optional) – the desired data type of returned tensor. If specified, the input tensor is casted to dtype before the operation is performed. This is useful for preventing data type overflows. Default: None. out (Tensor, optional) – the output tensor.

Code Examples

```
>>> a = torch.randn(1, 3)
>>> a
tensor([[ 0.2294, -0.5481,  1.3288]])
>>> torch.mean(a)
tensor(0.3367)
```

```
>>> a = torch.randn(4, 4)
>>> a
tensor([[-0.3841,  0.6320,  0.4254, -0.7384],
        [-0.9644,  1.0131, -0.6549, -1.4279],
        [-0.2951, -1.3350, -0.7694,  0.5600],
        [ 1.0842, -0.9580,  0.3623,  0.2343]])
>>> torch.mean(a, 1)
tensor([-0.0163, -0.5085, -0.4599,  0.1807])
>>> torch.mean(a, 1, True)
tensor([[-0.0163],
        [-0.5085],
        [-0.4599],
        [ 0.1807]])
```

Notes & Warnings

NOTE Note If the input tensor is empty, torch.mean() returns nan. This behavior is consistent with NumPy and follows the definition that the mean over an empty set is undefined.

NOTE See also torch.nanmean() computes the mean value of non-NaN elements.

Detailed Documentation

Documentation

If the input tensor is empty, torch.mean() returns nan. This behavior is consistent with NumPy and follows the definition that the mean over an empty set is undefined.

Returns the mean value of all elements in the input tensor. Input must be floating point or complex.

input (Tensor) – the input tensor, either of floating point or complex dtype

dtype (torch.dtype, optional) – the desired data type of returned tensor. If specified, the input tensor is casted to dtype before the operation is performed. This is useful for preventing data type overflows. Default: None.

Returns the mean value of each row of the input tensor in the given dimension dim. If dim is a list of dimensions, reduce over all of them.

If keepdim is True, the output tensor is of the same size as input except in the dimension(s) dim where it is of size 1. Otherwise, dim is squeezed (see torch.squeeze()), resulting in the output tensor having 1 (or len(dim)) fewer dimension(s).

input (Tensor) – the input tensor.

dim (int or tuple of ints, optional) – the dimension or dimensions to reduce. If None, all dimensions are reduced.

keepdim (bool, optional) – whether the output tensor has dim retained or not. Default: False.

dtype (torch.dtype, optional) – the desired data type of returned tensor. If specified, the input tensor is casted to dtype before the operation is performed. This is useful for preventing data type overflows. Default: None.

out (Tensor, optional) – the output tensor.

torch.nanmean() computes the mean value of non-NaN elements.